JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface MatchResult

All Known Implementing Classes:

Matcher

`public interface MatchResult`

The result of a match operation.

This interface contains query methods used to determine the results of a match against a regular expression. The match boundaries, groups and group boundaries can be seen but not modified through a `MatchResult`.

Implementation Note:

Support for named groups is implemented by the default methods `start(String)`, `end(String)` and `group(String)`. They all make use of the map returned by `namedGroups()`, whose default implementation simply throws `UnsupportedOperationException`. It is thus sufficient to override `namedGroups()` for these methods to work. However, overriding them directly might be preferable for performance or other reasons.

Since:

1.5

See Also:

Matcher

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
`int`	`end()`	Returns the offset after the last character matched.
`int`	`end(int group)`	Returns the offset after the last character of the subsequence captured by the given group during this match.
`default int`	`end(String name)`	Returns the offset after the last character of the subsequence captured by the given named-capturing group during

		the previous match operation.
String	**group**()	Returns the input subsequence matched by the previous match.
String	**group**(int group)	Returns the input subsequence captured by the given group during the previous match operation.
default String	**group**(String name)	Returns the input subsequence captured by the given named-capturing group during the previous match operation.
int	**groupCount**()	Returns the number of capturing groups in this match result's pattern.
default boolean	**hasMatch**()	Returns whether this contains a valid match from a previous match or find operation.
default Map<String, Integer>	**namedGroups**()	Returns an unmodifiable map from capturing group names to group numbers.
int	**start**()	Returns the start index of the match.
int	**start**(int group)	Returns the start index of the subsequence captured by the given group during this match.

default int	start(String name)	Returns the start index of the subsequence captured by the given named-capturing group during the previous match operation.

Method Details

start

```
int start()
```

Returns the start index of the match.

Returns:

The index of the first character matched

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

start

```
int start(int group)
```

Returns the start index of the subsequence captured by the given group during this match.

Capturing groups are indexed from left to right, starting at one. Group zero denotes the entire pattern, so the expression *m*.`start(0)` is equivalent to *m*.`start()`.

Parameters:

`group` - The index of a capturing group in this matcher's pattern

Returns:

The index of the first character captured by the group, or `-1` if the match was successful but the group itself did not match anything

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

`IndexOutOfBoundsException` - If there is no capturing group in the pattern with the given index

start

```
default int start(String name)
```

Returns the start index of the subsequence captured by the given named-capturing group during the previous match operation.

Implementation Requirements:

The default implementation of this method invokes `namedGroups()` to obtain the group number from the `name` argument, and uses it as argument to an invocation of `start(int)`.

Parameters:

`name` - The name of a named-capturing group in this matcher's pattern

Returns:

The index of the first character captured by the group, or `-1` if the match was successful but the group itself did not match anything

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

`IllegalArgumentException` - If there is no capturing group in the pattern with the given name

Since:

20

end

```
int end()
```

Returns the offset after the last character matched.

Returns:

The offset after the last character matched

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

end

```
int end(int group)
```

Returns the offset after the last character of the subsequence captured by the given group during this match.

Capturing groups are indexed from left to right, starting at one. Group zero denotes the entire pattern, so the expression *m*.`end(0)` is equivalent to *m*.`end()`.

Parameters:

`group` - The index of a capturing group in this matcher's pattern

Returns:

The offset after the last character captured by the group, or `-1` if the match was successful but the group itself did not match anything

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

`IndexOutOfBoundsException` - If there is no capturing group in the pattern with the given index

end

```
default int end(String name)
```

Returns the offset after the last character of the subsequence captured by the given named-capturing group during the previous match operation.

Implementation Requirements:

The default implementation of this method invokes `namedGroups()` to obtain the group number from the `name` argument, and uses it as argument to an invocation of `end(int)`.

Parameters:

`name` - The name of a named-capturing group in this matcher's pattern

Returns:

The offset after the last character captured by the group, or `-1` if the match was successful but the group itself did not match anything

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

`IllegalArgumentException` - If there is no capturing group in the pattern with the given name

Since:

20

group

```
String group()
```

Returns the input subsequence matched by the previous match.

For a matcher *m* with input sequence *s*, the expressions *m*.`group()` and *s*.`substring(`*m*.`start(), `*m*.`end())` are equivalent.

Note that some patterns, for example `a*`, match the empty string. This method will return the empty string when the pattern successfully matches the empty string in the input.

Returns:

The (possibly empty) subsequence matched by the previous match, in string form

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

group

`String group(int group)`

Returns the input subsequence captured by the given group during the previous match operation.

For a matcher *m*, input sequence *s*, and group index *g*, the expressions *m*.`group(`*g*`)` and *s*.`substring(`*m*.`start(`*g*` ), `*m*.`end(`*g*`))` are equivalent.

Capturing groups are indexed from left to right, starting at one. Group zero denotes the entire pattern, so the expression `m.group(0)` is equivalent to `m.group()`.

If the match was successful but the group specified failed to match any part of the input sequence, then `null` is returned. Note that some groups, for example `(a*)`, match the empty string. This method will return the empty string when such a group successfully matches the empty string in the input.

Parameters:

`group` - The index of a capturing group in this matcher's pattern

Returns:

The (possibly empty) subsequence captured by the group during the previous match, or `null` if the group failed to match part of the input

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

`IndexOutOfBoundsException` - If there is no capturing group in the pattern with the given index

group

`default String group(String name)`

Returns the input subsequence captured by the given named-capturing group during the previous match operation.

If the match was successful but the group specified failed to match any part of the input sequence, then `null` is returned. Note that some groups, for example `(a*)`, match the empty string. This method will return the empty string when such a group successfully matches the empty string in the input.

Implementation Requirements:

The default implementation of this method invokes `namedGroups()` to obtain the group number from the `name` argument, and uses it as argument to an invocation of `group(int)`.

Parameters:

`name` - The name of a named-capturing group in this matcher's pattern

Returns:

The (possibly empty) subsequence captured by the named group during the previous match, or `null` if the group failed to match part of the input

Throws:

`IllegalStateException` - If no match has yet been attempted, or if the previous match operation failed

`IllegalArgumentException` - If there is no capturing group in the pattern with the given name

Since:

20

groupCount

`int groupCount()`

Returns the number of capturing groups in this match result's pattern.

Group zero denotes the entire pattern by convention. It is not included in this count.

Any non-negative integer smaller than or equal to the value returned by this method is guaranteed to be a valid group index for this matcher.

Returns:

The number of capturing groups in this matcher's pattern

namedGroups

```
default Map<String,Integer> namedGroups()
```

Returns an unmodifiable map from capturing group names to group numbers. If there are no named groups, returns an empty map.

API Note:

This method must be overridden by an implementation that supports named groups.

Implementation Requirements:

The default implementation of this method always throws UnsupportedOperationException

Returns:

an unmodifiable map from capturing group names to group numbers

Throws:

UnsupportedOperationException - if the implementation does not support named groups.

Since:

20

hasMatch

```
default boolean hasMatch()
```

Returns whether `this` contains a valid match from a previous match or find operation.

Implementation Requirements:

The default implementation of this method always throws UnsupportedOperationException

Returns:

whether `this` contains a valid match

Throws:

UnsupportedOperationException - if the implementation cannot report whether it has a match

Since:

20

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.